

Validation techniques

A reasonableness check on input — not a truth check.

What this key knowledge covers

This summary distils everything you need for **KK11 — Validation techniques** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

01 What validation actually is

Validation is an automated check a program runs on input data to confirm it is **reasonable, sensible and allowable** before that data is processed or stored. It sits in the **development** stage of the problem-solving methodology, alongside coding, testing and documentation.

The key word is **reasonable**, not **correct**. Validation can tell that an age of 200 is impossible, or that a blank name is missing, or that the word "three" isn't a number. It cannot tell that someone who typed age 17 is actually 16 — that value is perfectly *reasonable*, just wrong. Catching wrong-but-reasonable data is a different job (verification — section 06).

Airport gate analogy. Validation is the gate agent who checks your boarding pass is *filled in* (existence), is a *boarding pass and not a receipt* (type), and is for a *flight that exists today* (range). They wave through anyone whose pass looks plausible. They do **not** check whether you're the actual person named on it — that's a different desk. A plausible pass can still belong to the wrong traveller.

Why bother? Computers follow the principle **GIGO — garbage in, garbage out**. If you let nonsense into the program, every calculation, sort and report built on it is also nonsense. Validation is the cheapest place to stop that: at the moment of input, before the bad data spreads. It improves **data integrity** and makes the solution more reliable — but it **reduces** errors, it never **eliminates** them.

3D Lab A · The Validation Gauntlet

A booking request travels along a conveyor toward CourtBook's database. It must clear **Existence**, then **Type**, then **Range**. Pick what the user typed in the **court number** field and watch which gate catches it. Order matters — the first failing gate stops it.

Confirms a required value has been entered and the field is not blank.

It says nothing about whether the value is the right type or sensible — only that *something* is there. Used on every field a record cannot do without.

In CourtBook: the *name* and *student ID* fields both need an existence check — a booking with no name is useless to the teacher reading the timetable. The check passes the moment any character is present and fails only on blank/whitespace.

Confirms the data entered matches the expected data type.

Numbers where a number is expected, a valid date where a date is expected, text where text is expected. Stops a word being treated as a number, which would crash a calculation.

In CourtBook: the *court number*, *duration* and *party size* fields all expect integers; the *date* field expects a valid calendar date. Typing G4 or "half an hour" fails the type check before any maths happens.

Confirms a value falls between an acceptable minimum and maximum (or within a limit).

Applies to numbers and dates. The value has already passed type checking — now we ask whether it's inside the sensible boundaries for this field.

In CourtBook: court number **1-6**, duration **15-60** minutes, party size **1-10**, and the booking date must be **today or later** (a date range). A range check is the only one of the three that needs you to know the real-world limits of the problem.

05 All three, side by side

Notice the order is also a logical pipeline: there's no point type-checking a blank, and no point range-checking text. Run them **existence** → **type** → **range** and each check can assume the previous one passed.

right kind

"Is it a number / date / text?"

within limits

"Is it between min and max?"

3D Lab B · Valid but wrong

CourtBook's **student ID** field wants a 5-digit number. The value 99999 exists, is the right type, and is in range — so it **passes validation**. But no student is enrolled with that ID, so it is **wrong**. Think of validation as throwing a dart at a board: landing *on the board* means plausible; hitting the *bullseye* means actually correct. Validation only checks the board.

07 Live form validator

Every field below runs existence, type and range checks as you type. Watch the message under each field name the check that fails. Then hit **Save booking** to see whether the whole record is accepted.

10 Command-word decoder

Give the answer only. "Name the validation check that confirms a field is not blank." → *Existence check*. No explanation needed.

Say what it is *and* show it in context. "Describe a range check for the duration field." → it confirms the value is between a min and max; here, 15–60 minutes.

Give the reason / how it works / why it matters. "Explain why validation can't guarantee correct data." → it tests plausibility, not truth; a reasonable value may still be wrong.

Make the difference explicit with "whereas". "Validation checks input is reasonable, *whereas* verification checks it is accurate against the true source."

11 P·E·L writing trainer

Draft a 3-mark answer to the prompt below. As you type, the keyword chips light up when you use the language an examiner rewards. Then reveal the model answer to compare.

12 VCE-style practice questions

Try each one on paper first. Then read the annotated weak answer, the full-mark answer, and the marking guide. Mark yourself honestly with the buttons — it feeds the dock.

Validation techniques — exam-day recall

- Confirms a value **has been entered** (not blank).
- Also called presence / required check.
- if name == "" then reject
- Says nothing about whether it's correct.
- Confirms the data is the **expected data type**.
- Number / date / text / Boolean.
- if not isInteger(x) then reject
- Stops "G4" being treated as a number.
- Confirms a value is **between min and max** / within a limit.
- Numbers and dates.
- if x < 1 or x > 6 then reject
- Derived values need **range**, not type.
- **Validation** = input is reasonable (as entered).
- **Verification** = input is accurate vs a true source.
- **Testing** = the program works (expected vs actual).
- Always **contextualise**: name field + boundaries.

Common traps & misconceptions

Exam trap

Trap: an existence check confirms **"not blank"** — nothing more. "asdf" passes the existence check on a name field. Don't write that existence checking "makes sure the name is correct" — it doesn't.

Exam trap

The classic VCAA trap. A **derived** value — one the program calculates, like *total court time* = *duration* × *sessions* — is already numeric, so it doesn't need a **type** check. To confirm it doesn't exceed the daily limit you need a **range** check. Students who answer "type check" here lose the mark. Always ask: *am I checking the kind of data, or the size of it?*

Exam trap

Carry this into the exam. "It passed validation" never means "it is correct." Validation = on the board. Correctness = the bullseye. The job that aims for the bullseye is **verification**.

Exam trap

Validation ≠ verification. Validation checks *plausibility* of input; verification checks it is *truthful/accurate*, usually by comparing against an authoritative source or a second entry. Valid data can still be unverified (and wrong).

Exam trap

Validation ≠ testing. Validation is checks the *program runs on the user's data*. Testing is the *developer checking the program* with deliberate test data and comparing expected vs actual results (that's KK12). Don't describe writing a test table as "validation."

Exam trap

1 · "Valid means correct." No — valid means reasonable. Always leave room for verification.

Exam trap

2 · Derived value → type check. A calculated number is already a number; check its **range** against the limit, not its type.

Exam trap

3 · Un-contextualised answers. "A range check" earns little. "A range check on duration to ensure it is 15–60 min" earns the mark. Name the field and the boundaries.

Exam trap

4 · Existence ≠ correctness. Existence only confirms "not blank." Don't claim it checks the value is right.

Exam trap

5 · Calling testing "validation." A test table with expected vs actual is **testing** (KK12), done by the developer — not validation of user input.

Exam trap

6 · "Validation removes all errors." It **reduces** error and improves integrity; it never guarantees error-free data.

Exam trap

7 · Wrong boundary on range. Watch $<$ vs $\leq$. "Court must be 1–6" means reject 0 and 7, accept 1 and 6. Off-by-one boundaries are a favourite of test-data questions.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Practice question · 3 marks

() — validation = automated reasonableness/allowability check on input (not “correct”). — performed before the data is processed/stored. — names all three: existence, type, range (all three required for the mark). Total:

✗ A weak answer

Validation is when you make sure the data is right before you use it. The three types are checking it's there, checking it and checking the numbers. **Why it loses marks:** Two real problems: “make sure the data is *right*” describes verification, not validation (validation checks reasonableness). And the three techniques are too vague to name marks — “checking it” isn't a technique.

✓ A strong answer

Validation is an automated check a program performs on input data to confirm it is **reasonable and allowable** before processing. The three techniques are **existence checking, type checking** and **range checking**.

Q2. Practice question · 4 marks

() — existence described + a blank rejected. — type described + a non-numeric value rejected. — range described as between 15 and 60. — a value outside 15–60 rejected (e.g. 5 or 90). Total:

✗ A weak answer

Existence so it's not empty. Type so it's a number. Range so it's not too big. It would reject 100. **Why it loses marks:** The descriptions are on the right track but only one rejecting value is given for all three, and “not too big” ignores the lower bound (15). Each technique needs its own rejected example to score fully.

✓ A strong answer

Existence — confirms a value was entered (rejects a blank field). **Type** — confirms it is a whole number (rejects “half hour”). **Range** — confirms it is between 15 and 60 minutes inclusive (rejects 5 or 90).

Q3. Practice question · 4 marks

() — validation = reasonableness of input, with example. — verification = accuracy vs a true source, with example. — clear contrast drawn between validation and verification (e.g. “whereas”). — testing = developer checks the solution works (expected vs actual), distinct from validating input. Total:

✗ A weak answer

Validation and verification both check the data. Testing is when you run the program to see if it works. **Why it loses marks:** It states they “both check the data” without drawing the distinction — that's the whole question. No examples, and the testing line, while not wrong, isn't tied to validation or to CourtBook.

✓ A strong answer

Validation checks input is **reasonable** as it is entered — e.g. CourtBook confirms the court number is 1–6. **Verification** checks input is **accurate** against a true source — e.g. confirming the entered student ID matches the enrolment list. **Testing** checks the **program itself works** — e.g. the developer feeds court 0 and 7 as test data and confirms both are rejected (expected vs actual).

Q4. Practice question · 4 marks

() — identifies the field as a derived/calculated value. — explains it is already numeric so a type check is redundant. — names range check as the correct technique. — contextualises it to the 200-minute cap. Total:

✗ A weak answer

It doesn't need a type check because it's fine. It needs validation to keep it under 200. **Why it loses marks:** “Because it's fine” explains nothing. It also never names the correct technique — “validation” is the category, not the specific check. The mark sits on saying *range* and *why not type*.

✓ A strong answer

Total weekly minutes is a **derived value** the program calculates from existing numbers, so it is already numeric — a **type check** would always pass and adds nothing. The correct technique is a **range check**: confirm the total is not greater than the 200-minute cap (e.g. if total > 200 then reject).